

PROJECT OVERVIEW

Lyrics-Based Semantic Song Search Engine

Find songs by describing their mood and emotional themes.

No title or artist required. Describe a feeling.

PROPOSED TECHNOLOGY STACK

Python

FastAPI

Sentence Transformers

Milvus

HNSW

PROPOSED SYSTEM

EXAMPLE QUERIES

"Lonely and reflective
after a breakup."

"Hopeful lyrics about
starting over."

DATASET AND SCOPE

Lyrics are the source of meaning.

Spotify Million Song Dataset from Kaggle

APPROXIMATE TRACK COUNT

57,650

before cleaning

spotify_millsongdata.csv

FIELD	CONTENTS
artist	Artist name
song	Song title
link	Source link or path
text	Song lyrics

SEARCHABLE THEMES

Sadness, hope, love, anger, nostalgia

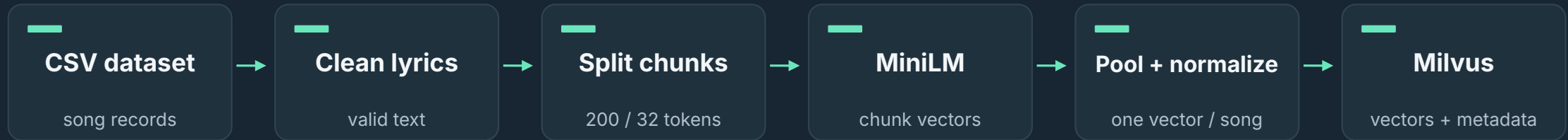
LYRICS, NOT AUDIO

No measured tempo, instruments or acoustic energy

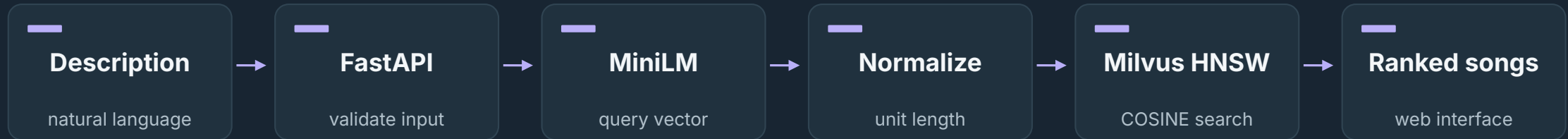
Prepare once. Search on demand.

Two workflows share one embedding model and one song collection.

OFFLINE / PREPARE THE COLLECTION



ONLINE / ANSWER A QUERY



Song vectors are prepared in advance. Each search embeds only the new query.

DATA PREPROCESSING

Clean records. Keep the meaning.

Conservative cleaning preserves emotional wording and reliable song identities.

01 / VALIDATE & CLEAN

- Check required fields
- Remove empty lyrics
- Normalize extra whitespace
- Remove exact duplicates

02 / PRESERVE MEANING

Keep negations and important wording.

"I love you."
"I do not love you."

Same title + different artist?
Keep both records.

03 / KEEP IDS ALIGNED

Assign a stable song ID.

song_id



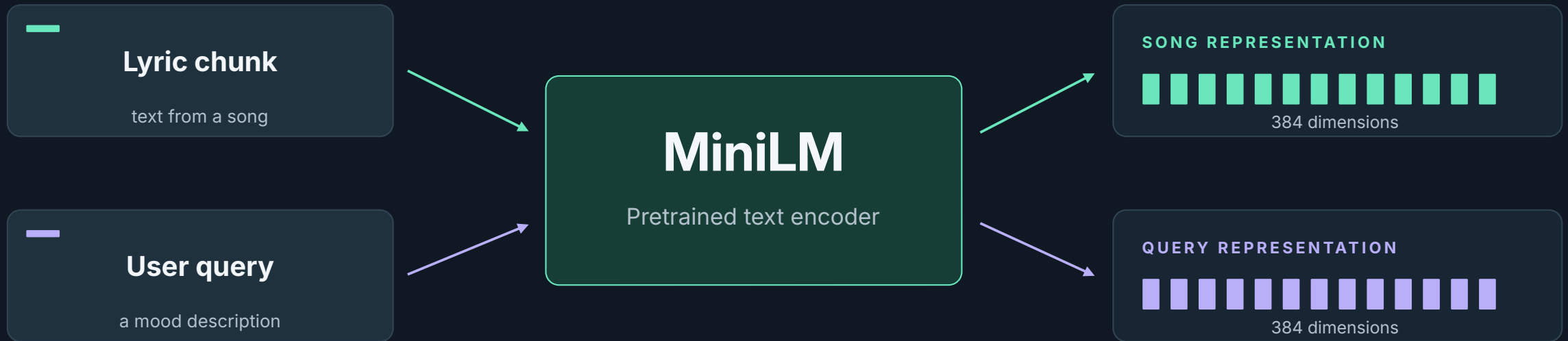
vector

Keep the correct artist,
title and link attached.

OUTPUT / A cleaned collection with every vector linked to the right song.

One shared space. 384 numbers.

sentence-transformers/all-MiniLM-L6-v2



USE THE PRETRAINED MODEL

No fine-tuning or weight updates.

KEEP ALL VALID SONGS SEARCHABLE

Separate development and final test queries.

LONG LYRICS: CHUNKING AND POOLING

Keep the whole lyric in view.

Default model limit: 256 word pieces. Leave room for special tokens.

PROPOSED CHUNKING

200 content tokens + 32-token overlap

Example: a song containing 450 content tokens

Chunk 1

1-200

Chunk 2

169-368

Chunk 3

337-450

POOLING

Many chunks. One song.

- 1 Normalize each chunk vector
- 2 Average matching coordinates
- 3 Normalize the average

1 song vector / 384 dimensions

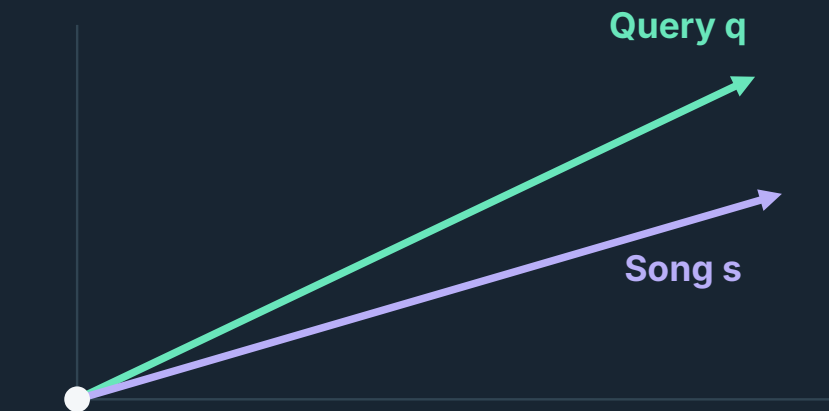
200 describes input length. 384 describes representation size.

SIMILARITY AND RANKING

Score the match. Rank the songs.

Cosine defines the comparison. Vector search selects the best-scoring candidates.

COMPARE VECTOR DIRECTIONS



Conceptual 2D illustration

UNIT-NORMALIZED VECTORS

$$\text{cosine}(q, s) = q \cdot s$$

Higher scores mean closer vector directions.

Return the highest-scoring matches first.

A similarity score is not a probability that a listener will enjoy the song.

VECTOR DATABASE AND HNSW INDEX

Milvus stores. HNSW navigates.

Proposed vector database and approximate-search index, using COSINE similarity.

STORE + RETRIEVE

MilvusClient

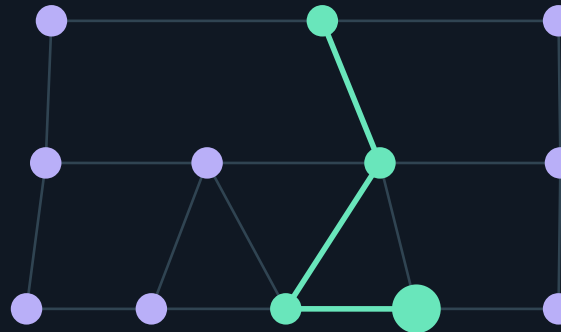
Python sends insert and search requests.



Milvus collection

ID + vector + artist + title + link

HNSW GRAPH



Follow links toward promising matches.

STARTING VALUES

M	16
efConstruction	200
ef	64
top_k	10

Not verified optimal settings.

TRADE-OFF / Potentially faster search; more graph memory and approximate results.

A simple search experience.

Enter a mood description, trigger a search, and view ranked song matches.

● ● ● INTERFACE CONCEPT / NOT A LIVE SCREENSHOT

Find your next lyrical mood.

Search

RANKED RESULTS		SIMILARITY
01	Song title Artist / optional short lyrics excerpt	--
02	Song title Artist / optional short lyrics excerpt	--

Results are based on lyrics and inferred themes, not audio analysis.

PLANNED ENDPOINTS

GET /

Search page

POST /search

Ranked matches

GET /health

Engine readiness

Loading, error and
empty-result states

EVALUATION AND CONCLUSION

Evaluate relevance and performance.

Proposed evaluation plan: report measurements only after execution.

QUERY-BASED EVALUATION

10

Development queries

20

Final test queries

Human relevance grades: 0 / 1 / 2

Not relevant / partially relevant / strongly relevant

SEARCH QUALITY

Precision@5 + nDCG@10

PERFORMANCE

Query embedding + vector-search latency
Build time + memory use

Natural-language mood → lyrical meaning → ranked songs

LIMITS Lyrics do not establish audio features; pooling can blur contrasting emotions.

STATUS Milvus/HNSW deployment and performance remain unverified.